

Aero 431 Final Project: Wing Bending Study

Zach Selzman,*Nathan Beifuss,*Leah Loftus,*

Olivia Boulcy,*Liv Zuhoski,*

California Polytechnic State University, San Luis Obispo, CA, 93402

A semi-monocoque wing structure was designed and constructed to evaluate structural performance under the conditions of a cantilever with applied static and dynamic load at the tip. The wing consisted of 3D printed ribs, two wooden spars, and a carbon fiber layup wrapped around. The wing was analyzed using beam theory FEM methods learned in class to estimate the failure that would occur due to bending stress and tip deflection. Analytical calculations predicted the wing, modeled as a beam then plate, would not fail from deflection when a factor of safety of two was applied to the spar bending strength, although deflection calculations gave a higher theoretical limit. Experimental testing was performed by clamping the wing and, starting with 1 lb, applying incremental loads to the tip of the wing until failure. This wing failed at 5 lb static load when the deflection of the wing exceeded half the span before the dynamic load was applied. The results from this demonstrate that simplified beam models can reasonably predict structural behavior while displaying the influence of material variability and manufacturing effects on experimental performance.

Nomenclature

D	=	Laminate bending stiffness matrix
E	=	Modulus of elasticity
FEM	=	Finite element method
Hz	=	Hertz
I	=	Second moment of area
K	=	Global stiffness matrix
lb	=	Pound
M	=	Bending moment
M	=	Consistent mass matrix
m	=	Meters
mm	=	Millimeters
N	=	Newtons
w	=	Transverse deflection
x	=	Spanwise coordinate
y	=	Distance from neutral axis
σ	=	Bending stress
ϕ	=	Mode shape vector
θ	=	Ply angle
ω	=	Angular frequency
PLA	=	Polylactic acid (material used for 3D printing)

I. Introduction

THE structural integrity of an aircraft is a fundamental requirement in the design process. A wing must support aerodynamic, inertial and ground loads throughout its lifetime without failing or deforming to an inoperable point. Understanding how wings respond to these loads, such as where stresses concentrate, how deflection grows and the point at which it fails is integral to aircraft design. Even on a smaller scale, like in this experiment, the governing mechanics

*Student, Department of Aerospace Engineering, 1 Grand Ave, San Luis Obispo, CA, 93402.

still mirror those of a full-scale flight structure, which makes testing simplified models an incredibly valuable tool for analytics and closing the gap between predictions and real-life behavior.

The modern aircraft wing is generally constructed using composite materials, and carbon fiber reinforced polymer laminates have become increasingly popular choices in commercial and military applications. Carbon fiber provides a higher specific stiffness and specific strength than aluminum, which makes for a lighter structure able to hold heavier loads. The directional characteristics of carbon fiber allow engineers to change the laminate layup to stiffen the structure against any chosen loading modes. This tunability is a large advantage over isotropic metals but also brings with it some complexities. The response to stress is highly dependent on the laminate configuration, so some simplified isotropic models cannot accurately predict the behavior of a composite structure.

Many analytical models used to predict structural behavior, rely on pre-defined simplifying assumptions. Treating the wing as a beam assumes that shear deformation is negligible and that the structure is homogeneous within a cross-section. In the real world, a semi-monocoque wing with spars and ribs in discrete areas along a composite material is more complex than a beam. Discontinuities in this internal structure can create local stresses that are not seen in a beam model. These issues make physical testing all the more important.

In this project, a semi-monocoque wing was designed, manufactured and tested to observe the structural performance under a cantilever tip load. Analytical models, simple and complex, were used to predict the tip deflection and failure load, which were compared with experimental values.

II. Background

The usual structural behavior of a cantilevered wing under a load on the tip is to bend along the span. This behavior can be approximated using the Euler-Bernoulli beam theory. In the model, the wing is treated as a slender beam, fixed at one end and the relationship between moment M , curvature and bending stiffness can be observed through the equation

$$EI \frac{d^2 w}{dx^2} = M(x) \quad (1)$$

and the bending stress at any given point in a cross section is calculated as

$$\sigma = \frac{My}{I} \quad (2)$$

with this max bending stress occurring at the fixed end of the beam, which is also where the moment is greatest.

Numerically solving for the deflection profile requires the use of a finite element method. The wingspan is split into n parts and over each component/element, the deflection is estimated by a smooth function which matches the displacement and slope at each element's boundaries. A stiffness matrix is then assembled for each element based on the bending stiffness. Ultimately a global stiffness matrix is created and used to solve the complete deflection profile across the span.

The final analysis incorporated the carbon fiber laminate, and the wing is modeled as a plate rather than a beam. A laminate bending stiffness matrix D was computed from the fiber orientations and ply thicknesses of the layup. The Rayleigh-Ritz method was then applied, which approximated the deflection as a linear combination of basis functions. The goal is to minimize the total potential energy.

The natural frequency of the wing was also approximated using modal analysis, solving the eigenvalue general equation

$$K\phi = \omega^2 M\phi \quad (3)$$

for the angular frequencies and mode shapes of which they correspond. This method can be used to find the fundamental bending frequency, which relates to the dynamic loading.

III. Procedure

The principle objective of this project was to make the lightest possible wing that adhered to the requirements. Carbon fiber represented an optimal path for reducing weight without compromising on strength, while providing a relevant and engaging manufacturing challenge. To manufacture the wing, a carbon fiber shell was selected. To achieve the precise geometry of the airfoil, two molds were used to manufacture a top and bottom skin. The wing's composite skin was manufactured from separate upper and lower surfaces for a few key reasons. Firstly, ribs needed to be able to be placed and bonded inside the upper and lower skins, this would have been difficult if it was one solid piece, however

there would be a benefit to this as the leading edge and trailing edge would not need to be bonded. Even though the final skins were laid inside molds of the negative, thought was put into laying up the carbon around a positive which would be removed after it cured. While this could have worked, it's possible that the peel ply finish would not have been precise enough, and the airfoil shape would not be accurate. Lastly, the airfoil was not laid up inside of a one piece negative mold, with a "donut bag" because it would have been too complex with the materials available.

A. Mold Manufacturing

The mold was designed in solidworks by making a negative of the wing, and splitting that into two halves forming the upper and lower skin molds. In a larger scale project, instead of having two symmetrical skin molds, a joggle could be used to increase the strength of the leading edge and to increase the bond area between the two skins. Since the mold is a true negative of the geometrically accurate wing, the resulting skins matched the exact geometry of the wing model according to the objective. The top and lower skin molds were split into 6 parts that could be joined by dowels in order to fit on the 10 in 3d print bed.

It was decided that the molds must be 3d printed for a few reasons. The foremost reason was price and accessibility. 3D printing was free through mustang 60. The other option would have been using CNC cut high density foam, and while this does have the upside of being Pre-Preg compatible, the foam would have to be ordered online and no one in the group has CNC training.

Once all 6 pieces of the molds had been printed, they were joined with 3d printed dowels and epoxy. To attain the best finish possible, the molds were heavily sanded, and resin coated. Instead of the traditional route of using peelply to prevent adhesion, the mold was waxed 5 times to prevent sticking.

B. Layup

The actual part was developed using the wet layup technique. Ultimately, five layers of carbon fiber were coated with epoxy resin before being placed inside of a vacuum bag. The wing weight was the principle concern, thus, the amount of resin was carefully measured. The layup sequence was as follows:

$$\theta = [-45, 45, 0, 90, \dots, 0, -90] \quad (4)$$

Where the wing was designed to have two sheet layers (each layer has two unidirectional weaves) on the top surface and one on the bottom surface. This was done to reduce weight, in addition to the knowledge that reinforcement on the tension-pulling top surface would be critical. The all up fiber weight was approximately 55 grams. As such, a rough epoxy calculation was performed using the following:

$$\text{Epoxy Weight [grams]} = 1.5 \times \text{Fiber Weight} \quad (5)$$

However, in efforts to save weight, the total amount of epoxy applied was less than 70 grams. Consequently, the laminate, when released from the mold (shown in Figure 3b)) was far dryer than expected. Additionally, without heat and a proper vacuum setup, the total epoxy percolation throughout the fibers was worse than expected—illustrating the manufacturing challenges. The poor fiber saturation resulted in subpar stiffness, however, due to time constraints, we lacked the ability to reiterate. In the future, a positive mold made from ABS would be better, as a stronger vacuum can be achieved in addition to heat-assisted curing in an autoclave. However, as insurance, the ribs were engineered to be compatible with wood spar dowels if need be, although this adds significant weight to the overall structure—nearly doubling the all-up wing weight to over 200 grams. Regardless, our group's wing was one of the lighter structures in the test pool.

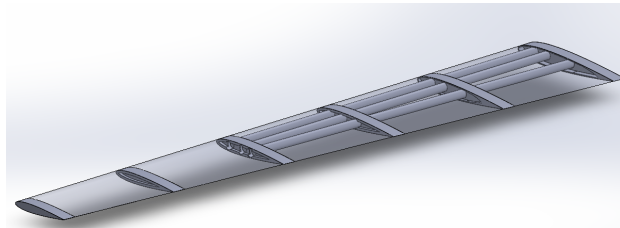
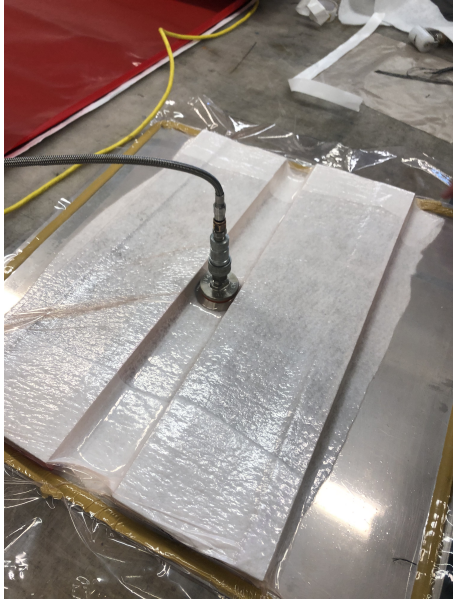


Fig. 1 Wing Assembly



a) Vacuuming the carbon to the molds



b) Joining the top and bottom skin

Fig. 2 Molding and skin joining processes



a) Dremeling the top skin to shape



b) Joining the top and bottom skin

Fig. 3 Final shaping and assembly of the wing component



a) Wing with dowels



b) Wing assembled with clamp

Fig. 4 Final assembly

IV. Analysis

The main goal of the analysis was to predict the wing deflection and root bending stress under the required loading conditions. The wing was designed to sustain both static and dynamic loads ranging from 1 lb to 5 lb. Wing failure we focused on was defined by deflection of more than half span or total breakage.

Before any manufacturing had taken place and the layup had not been finalized, the first analysis model treated the wing as a simple Euler-Bernoulli beam. This was the first calculation made assuming the main structural response during testing is suspected to be bending along the span.

The wing was analyzed using a FEM beam code based on the beam finite element method from class. The code divides the wing span into elements and solves for the vertical deflection along the wing with a static load was applied at the free end. From the code, the tip load of 5 lb would create a tip deflection of 20.019 mm, as seen in Fig. 5. The estimated static failure load is calculated to be 28.20 lb. Therefore, based on our preliminary model, the wing is expected to survive the required 5 lb static load.

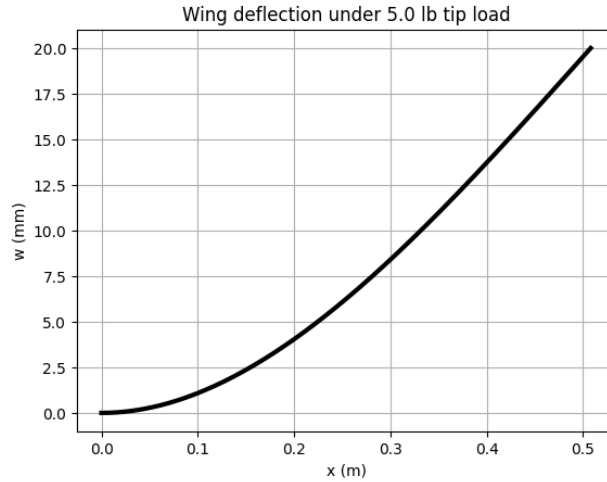


Fig. 5

After manufacturing had taken place and the layup was finalized, the main analysis was performed. By approximating the carbon fiber layup as the wing material, with thickness taken into account, due to it being greatly outweighing the spars and ribs in terms of stiffness and strength. The layup degrees were $[0, 90, 45, 45, 45]$, with one layer on the bottom and two layers on the top of the wing. At a 5 lb static load place at the center of the tip, the wing tip deflection was calculated as 2.674 mm. The full deflection profile of the wing can be seen in Fig. 6. below. This was calculated using the FRC library functions and code that can be found in the appendix.

Carbon-Epoxy Wing Deflection at 5 lbs

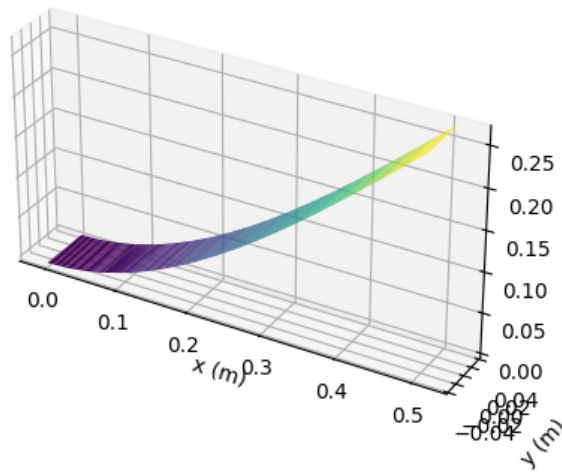


Fig. 6

Lastly, to round out our calculations, the first fundamental frequency was calculated using the modal analysis code. This helped us in basing an argument for the dynamic loading on the wing. The wing's first fundamental frequency is 6.033479143198291 Hz, although the next step would be analyzing the time dependent response, which is out of the scope of the course content so far.

From the tip deflection, bending stress, and frequency calculations, we believe that our wing will pass the static and potentially the dynamic load. We were not taught fully how to calculate a dynamic load, so we are hoping that the low frequency and high yield strength from the static load will guarantee the success of our wing under a dynamic load.

V. Results and Discussion



Fig. 7 Testing the setup!

The final wing consisted of a carbon fiber primary structure reinforced with four 3D-printed ribs and two wooden dowel spars. The spars passed through two of the ribs and served as the primary load-carrying members. This is why the initial calculations did not align with the physical behavior of the wing. The original analysis considered the wing as a single structure, with spars going all the through. Static load testing was conducted by applying incremental loads at the wing tip and measuring the resulting deformation. As the applied load increased, the wing exhibited progressively larger tip deflections while maintaining its structural integrity at lower load levels. At the maximum test load of 5 lb, the wing experienced a tip deflection of approximately 10 inches while having bending deformation. The largest deformation occurred near the second rib from the root of the wing. The wing essentially snapped where the spars ended and was unable to maintain its structural shape under the applied load. Therefore considered to have failed due to excessive deformation.

In future testing, the wing structure should be modified by adding additional spars, or longer spars to increase its bending stiffness and improve its load-carrying capability. The additional spars will provide greater structural support and reduced likelihood of excessive deformation under future loading conditions.

VI. Conclusion

In aerospace applications it is necessary to understand under what loads and conditions a structure will fail or deflect enough to be inoperable. The prototype wing structure analyzed was designed as a semi-monocoque made up of 3D printed PLA airfoil ribs, two wooden spars, and a carbon fiber layup wrap. It was designed based on aerospace structural principles learned in AERO 431, including Rayleigh-Ritz method, modal analysis, natural frequencies, and finite element method. In addition to theoretical considerations, the availability, cost of materials and ease of manufacturing impacted the final configuration. Initial calculations were performed to determine expected failure, in which the team determined that tip deflection at 5 lb would not be enough to cause the wing to fail under static load. The team also performed small

load tests to make sure the wing was stable as part of the testing and verification process, which ended with the decision to add a second spar. In the final test outlined in the procedure, the wing failed at 5 lbs. The group hypothesizes that the lower failure weight occurred due to the spars not extending the full span of the wing, which was not accounted for in coding calculations. This investigation demonstrates the importance of aerospace structural airfoil design based on both strength and stiffness for performance under standard operational aerodynamic loading.

Appendix

A. Python Code

The following Python code was used to perform the finite element beam analysis for the wing under a 5 lb tip load.

```

1 # Finite Element Method for Beams
2 # AERO 431 (Spring 2026)
3 #
4 # Dr. Amuthan Ramabathiran
5 # aramabat@calpoly.edu
6
7 import numpy as np
8 import matplotlib.pyplot as plt
9
10 # Constants
11 E = 10e9 # Pa, epoxy modulus estimate
12 L = 0.508 # m, 20 in
13
14 root_chord = 4 / 39.37 # m
15 tip_chord = 2 / 39.37 # m
16 c_avg = 0.5 * (root_chord + tip_chord)
17
18 t = 0.009144 # thickness, m
19 # (4 + 2) / 2 * 0.12 = average thickness in inches,
20 # then divide by 39.37 to convert to meters.
21
22 # Tip load
23 P_lb = 5.0
24 P = P_lb * 4.44822 # N
25
26 I = c_avg * t**3 / 12
27
28 # Definition of shape functions
29 def Nc1(idx, x, xi, xj):
30     h = xj - xi
31     xx = (x - xi) / h
32
33     if idx == 0:
34         return 1 - 3 * xx**2 + 2 * xx**3
35     elif idx == 1:
36         return h * (xx - 2 * xx**2 + xx**3)
37     elif idx == 2:
38         return 3 * xx**2 - 2 * xx**3
39     elif idx == 3:
40         return h * (xx**3 - xx**2)
41
42 def d2Nc1(idx, x, xi, xj):
43     h = xj - xi
44     xx = (x - xi) / h
45
46     if idx == 0:

```

```

47     return (-6 + 12 * xx) / h**2
48 elif idx == 1:
49     return (-4 + 6 * xx) / h
50 elif idx == 2:
51     return (6 - 12 * xx) / h**2
52 elif idx == 3:
53     return (-2 + 6 * xx) / h
54
55 # Plot beam deflection
56 def plot_w(W, x_mesh):
57     n_elements = len(x_mesh) - 1
58     xs = np.linspace(x_mesh[0], x_mesh[-1], 200)
59     ws = np.zeros_like(xs)
60
61     for i, xval in enumerate(xs):
62         e = np.searchsorted(x_mesh, xval) - 1
63         e = max(0, min(e, n_elements - 1))
64
65         xi = x_mesh[e]
66         xj = x_mesh[e + 1]
67
68         ws[i] = (
69             W[2 * e] * Nc1(0, xval, xi, xj)
70             + W[2 * e + 1] * Nc1(1, xval, xi, xj)
71             + W[2 * (e + 1)] * Nc1(2, xval, xi, xj)
72             + W[2 * (e + 1) + 1] * Nc1(3, xval, xi, xj)
73         )
74
75     plt.plot(xs, 1000 * ws, 'k-', lw=3)
76     plt.xlabel('x (m)')
77     plt.ylabel('w (mm)')
78     plt.title(f'Wing deflection under {P_lb:.1f} lb tip load')
79     plt.grid()
80     plt.show()
81
82     return xs, ws
83
84 # Mesh
85 n_elements = 20
86 x_mesh = np.linspace(0, L, n_elements + 1)
87 h = L / n_elements
88
89 # Stiffness matrix
90 K = np.zeros((2 * n_elements, 2 * n_elements))
91
92 for i in range(n_elements - 1):
93     K[2 * i, 2 * i] = 24
94     K[2 * i, 2 * i + 2] = -12
95     K[2 * i, 2 * i + 3] = 6 * h
96
97     K[2 * i + 1, 2 * i + 1] = 8 * h**2
98     K[2 * i + 1, 2 * i + 2] = -6 * h
99     K[2 * i + 1, 2 * i + 3] = 2 * h**2
100
101 K[2 * n_elements - 2, 2 * n_elements - 2] = 12
102 K[2 * n_elements - 2, 2 * n_elements - 1] = -6 * h
103 K[2 * n_elements - 1, 2 * n_elements - 1] = 4 * h**2
104
105 for i in range(2 * n_elements):

```



```

106     for j in range(i):
107         K[i, j] = K[j, i]
108
109 K = (E * I / h**3) * K
110
111 # Load vector
112 # Static load at the tip
113 F = np.zeros(2 * n_elements)
114 F[2 * n_elements - 2] = P
115
116 W_unknown = np.linalg.solve(K, F)
117
118 W = np.hstack(([0.0, 0.0], W_unknown))
119
120 tip_deflection = W[-2]
121
122 print(f"Tip load = {P_lb:.2f} lb = {P:.2f} N")
123 print(f"Tip deflection = {1000 * tip_deflection:.3f} mm")
124
125 # Estimated failing load
126 sigma_allow = 60e6 # Pa, epoxy
127
128 M_root = P * L
129 y_max = t / 2
130 sigma_root = M_root * y_max / I
131
132 P_fail = sigma_allow * I / (L * y_max)
133 P_fail_lb = P_fail / 4.44822
134
135 print(f"Root bending stress at {P_lb:.1f} lb = {sigma_root / 1e6:.2f} MPa")
136 print(f"Estimated static failure load = {P_fail_lb:.2f} lb")
137
138 _ = plot_w(W, x_mesh)

```

Listing 1 Finite Element Beam Deflection Code

The following Python code was used to perform the carbon-epoxy laminate deflection analysis.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib import cm
4 import sympy
5
6 # If using the FRC library, make sure this import matches your file/library name:
7 # from frc import Laminate
8
9 theta = [0, 90, 45, 45, 45, 45]
10
11 E1 = 140e9
12 E2 = 10e9
13 nu12 = 0.3
14
15 t = 0.009144 # m, average thickness
16
17 laminate = Laminate(theta, t=t)
18
19 D = laminate.D
20
21 E = 3e9 # Pa, epoxy modulus estimate
22 rho = 2.7e3 # kg/m^3

```

```

23 root_chord = 4 / 39.37 # m
24 tip_chord = 2 / 39.37 # m
25
26
27 avg_chord = 0.5 * (root_chord + tip_chord)
28
29 l = 0.508 # m, 20 inches
30
31 I = avg_chord * t**3 / 12
32 A = avg_chord * t
33
34 # Constants
35 a = l
36 b = avg_chord
37
38 P = 22.24 # N, equivalent to 5 lb
39
40 # Symbols
41 x = sympy.symbols('x')
42 y = sympy.symbols('y')
43
44 # Rayleigh-Ritz method
45 def a_del_Pi(w, del_w):
46     w_x = sympy.diff(w, x)
47     w_y = sympy.diff(w, y)
48
49     w_xx = sympy.diff(w_x, x)
50     w_xy = sympy.diff(w_x, y)
51     w_yy = sympy.diff(w_y, y)
52
53     del_w_x = sympy.diff(del_w, x)
54     del_w_y = sympy.diff(del_w, y)
55
56     del_w_xx = sympy.diff(del_w_x, x)
57     del_w_xy = sympy.diff(del_w_x, y)
58     del_w_yy = sympy.diff(del_w_y, y)
59
60     integrand = (
61         D[0, 0] * w_xx * del_w_xx
62         + D[1, 1] * w_yy * del_w_yy
63         + D[0, 1] * (w_xx * del_w_yy + w_yy * del_w_xx)
64         + 4 * D[2, 2] * w_xy * del_w_xy
65         + D[0, 2] * (w_xx * del_w_xy + w_xy * del_w_xx)
66         + D[1, 2] * (w_yy * del_w_xy + w_xy * del_w_yy)
67     )
68
69     return sympy.integrate(
70         sympy.integrate(
71             integrand, (y, -b / 2, b / 2)
72         ),
73         (x, 0, a)
74     )
75
76 def l_del_Pi(del_w):
77     return P * del_w.subs([(x, a), (y, 0.0)])
78
79 # Basis functions
80 phi = [
81     x**2,

```

```

82     x**3,
83     x**4,
84     x**5
85 ]
86
87 n_basis = len(phi)
88
89 # Stiffness matrix
90 K = np.zeros((n_basis, n_basis))
91
92 for i in range(n_basis):
93     for j in range(i, n_basis):
94         K[i, j] = a_del_Pi(phi[i], phi[j])
95
96 for i in range(n_basis):
97     for j in range(i):
98         K[i, j] = K[j, i]
99
100 # Load vector
101 F = np.zeros(n_basis)
102
103 for i in range(n_basis):
104     F[i] = l_del_Pi(phi[i])
105
106 # Solve Kc = F
107 coefficients = np.linalg.solve(K, F)
108
109 print("Coefficients:")
110 for i in range(n_basis):
111     print(coefficients[i])
112
113 # Post-process solution
114 w = coefficients[0] * phi[0]
115
116 for i in range(1, n_basis):
117     w += coefficients[i] * phi[i]
118
119 f_w = sympy.lambdify((x, y), w, "numpy")
120
121 print(f"Tip deflection = {1000 * f_w(a, b / 2):.3f} mm")
122
123 # Plot solution
124 xs = np.linspace(0, a, 20)
125 ys = np.linspace(-b / 2, b / 2, 20)
126 xs, ys = np.meshgrid(xs, ys)
127 ws = f_w(xs, ys)
128
129 fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
130 ax.plot_surface(xs, ys, 100 * ws, cmap=cm.viridis, lw=0)
131 ax.set_xlabel("x (m)")
132 ax.set_ylabel("y (m)")
133 ax.set_zlabel("w (cm)")
134 ax.set_title("Carbon-Epoxy Wing Deflection at 5 lb")
135 ax.set_box_aspect((np.ptp(xs), np.ptp(ys), np.ptp(100 * ws)))
136
137 plt.show()

```

Listing 2 Beam Modal Analysis Code

The following Python code was used to perform fundamental frequency analysis.

```

1 x = sp.symbols('x')
2
3
4 E = 3e9 # Pa, epoxy modulus estimate
5 rho = 2.7e3 # kg/m^3
6
7 root_chord = 4 / 39.37 # m
8 tip_chord = 2 / 39.37 # m
9
10 c = 0.5 * (root_chord + tip_chord)
11
12 t = 0.009144 # m, average thickness
13 l = 0.508 # m, 20 inches
14
15 I = c*t**3/12
16 A = c*t
17
18 def a_beam(basis, i, j):
19     phi_i = basis[i]
20     phi_i_x = sp.diff(phi_i, x)
21     phi_i_xx = sp.diff(phi_i_x, x)
22
23     phi_j = basis[j]
24     phi_j_x = sp.diff(phi_j, x)
25     phi_j_xx = sp.diff(phi_j_x, x)
26
27     integrand = E * I * phi_i_xx * phi_j_xx
28     kij = sp.integrate(integrand, (x, 0, l))
29
30     return float(kij.evalf())
31
32 def mu_beam(basis, i, j):
33     phi_i = basis[i]
34     phi_j = basis[j]
35
36     integrand = rho * A * phi_i * phi_j
37     mij = sp.integrate(integrand, (x, 0, l))
38
39     return float(mij.evalf())
40
41 def compute_fundamental_frequency(basis):
42     n_basis = len(basis)
43
44     K = np.zeros((n_basis, n_basis))
45     for i in range(n_basis):
46         K[i,i] = a_beam(basis, i, i)
47         for j in range((i + 1), n_basis):
48             K[i, j] = a_beam(basis, i, j)
49             K[j, i] = K[i, j]
50
51     M = np.zeros((n_basis, n_basis))
52     for i in range(n_basis):
53         M[i,i] = mu_beam(basis, i, i)
54         for j in range((i + 1), n_basis):
55             M[i, j] = mu_beam(basis, i, j)
56             M[j, i] = M[i, j]
57
58     freqs, modes = eig(K, M)

```

```

59     return freqs, modes
60
61
62 def compute_w(basis, c):
63     n_basis = len(basis)
64     w = c[0] * basis[0]
65     for i in range(1, n_basis):
66         w += c[i] * basis[i]
67     return w
68
69 def plot_mode(basis, c, n=50):
70     xs = np.linspace(0, 1, n)
71
72     w = compute_w(basis, c)
73     ws = np.array([
74         w.subs(x, x_) for x_ in xs
75     ])
76
77     plt.plot(xs, ws, 'k-', lw=3)
78     plt.xlabel('x (m)')
79     plt.ylabel('w (m)')
80     plt.grid()
81     plt.show()
82
83
84 def animate_mode(basis, c, omega, n=50, T=10.0, nT = 1000):
85     ts = np.linspace(0, T, nT)
86     xs = np.linspace(0, 1, n)
87
88     w = compute_w(basis, c)
89     ws_mode = np.array([float(w.subs(x, x_)) for x_ in xs])
90
91     wmax = np.max(np.abs(ws_mode))
92
93     fig, ax = plt.subplots()
94     ln, = plt.plot([], [], 'k-', lw=3)
95     ax.set_xlim(0, 1)
96     ax.set_ylim(-1.2*wmax, 1.2*wmax)
97     ax.grid()
98
99     def update(i_frame):
100         ws = ws_mode * np.cos(omega*ts[i_frame])
101         ln.set_data(xs, ws)
102         return ln,
103
104     anim = FuncAnimation(
105         fig,
106         update,
107         frames=len(ts),
108         interval=200,
109         blit=True
110     )
111
112     plt.show()
113
114
115 def dynamic_mode_expansion(basis, modes, omegas, c, n=50, T=10.0, nT = 1000):
116     ts = np.linspace(0, T, nT)
117     xs = np.linspace(0, 1, n)

```

```

118 n_modes = modes.shape[1]
119 w_modes = np.zeros((n_modes, n))
120 for i_mode in range(n_modes):
121     w = compute_w(basis, modes[:,i_mode])
122     w_modes[i_mode,:] = np.array([float(w.subs(x, x_)) for x_ in xs])
123
124
125 fig, ax = plt.subplots(3, 2, figsize=(14,8))
126
127 ln0, = ax[0,0].plot([], [], 'k-', lw=3)
128 ln1, = ax[1,0].plot([], [], 'k-', lw=3)
129 ln2, = ax[2,0].plot([], [], 'k-', lw=3)
130 ln3, = ax[0,1].plot([], [], 'k-', lw=3)
131 ln4, = ax[1,1].plot([], [], 'k-', lw=3)
132 lnc, = ax[2,1].plot([], [], 'k-', lw=3)
133
134 w0max = np.max(np.abs(w_modes[0]))
135 ax[0,0].set_xlim(0, 1)
136 ax[0,0].set_ylim(-1.2*w0max, 1.2*w0max)
137 ax[0,0].grid()
138 ax[0,0].set_title('Mode 1')
139
140 w1max = np.max(np.abs(w_modes[1]))
141 ax[1,0].set_xlim(0, 1)
142 ax[1,0].set_ylim(-1.2*w1max, 1.2*w1max)
143 ax[1,0].grid()
144 ax[1,0].set_title('Mode 2')
145
146 w2max = np.max(np.abs(w_modes[2]))
147 ax[2,0].set_xlim(0, 1)
148 ax[2,0].set_ylim(-1.2*w2max, 1.2*w2max)
149 ax[2,0].grid()
150 ax[2,0].set_title('Mode 3')
151
152 w3max = np.max(np.abs(w_modes[3]))
153 ax[0,1].set_xlim(0, 1)
154 ax[0,1].set_ylim(-1.2*w3max, 1.2*w3max)
155 ax[0,1].grid()
156 ax[0,1].set_title('Mode 4')
157
158 w4max = np.max(np.abs(w_modes[4]))
159 ax[1,1].set_xlim(0, 1)
160 ax[1,1].set_ylim(-1.2*w4max, 1.2*w4max)
161 ax[1,1].grid()
162 ax[1,1].set_title('Mode 5')
163
164 wmax = np.max(np.abs(w_modes))
165 ax[2,1].set_xlim(0, 1)
166 ax[2,1].set_ylim(-1.2*wmax, 1.2*wmax)
167 ax[2,1].grid()
168 ax[2,1].set_title('Random combination')
169
170 def update_panel(i_frame):
171     ws_0 = w_modes[0] * np.cos(omegas[0]*ts[i_frame])
172     ws_1 = w_modes[1] * np.cos(omegas[1]*ts[i_frame])
173     ws_2 = w_modes[2] * np.cos(omegas[2]*ts[i_frame])
174     ws_3 = w_modes[3] * np.cos(omegas[3]*ts[i_frame])
175     ws_4 = w_modes[4] * np.cos(omegas[4]*ts[i_frame])
176     ws = c[0]*ws_0 + c[1]*ws_1 + c[2]*ws_2 + c[3]*ws_3 + c[4]*ws_4

```



```

177     ln0.set_data(xs, ws_0)
178     ln1.set_data(xs, ws_1)
179     ln2.set_data(xs, ws_2)
180     ln3.set_data(xs, ws_3)
181     ln4.set_data(xs, ws_4)
182     lnc.set_data(xs, ws)
183
184
185     return ln0, ln1, ln2, ln3, ln4, lnc
186
187 anim = FuncAnimation(
188     fig,
189     update_panel,
190     frames=len(ts),
191     interval=100,
192     blit=True
193 )
194
195 plt.show()
196
197
198 if __name__ == '__main__':
199     # Two modes model
200     basis = [
201         x**2,
202         x**3,
203         x**4,
204         x**5,
205         x**6
206     ]
207
208     # cantilever beam basis functions with  $w(0) = w'(0) = 0$ 
209
210     omega2s, modes = compute_fundamental_frequency(basis)
211
212     i_modes = np.argsort(omega2s.real)
213     omegas = np.sqrt(omega2s[i_modes].real)
214     modes[:, :] = modes[:, i_modes]
215
216     print(f'Fundamental Frequency = {omegas[0]/(2*np.pi)} Hz')
217
218     i_mode = 0
219     plot_mode(basis, modes[:, i_mode])
220
221     animate_mode(basis, modes[:, i_mode], np.sqrt(omega2s[i_mode].real))
222
223     # n_modes = modes.shape[1]
224     # c = np.random.random(n_modes)
225     # dynamic_mode_expansion(basis, modes, omegas, c)

```

Listing 3 Finite Element Beam Deflection Code